Bakery Ledger: Integrated Accounting Project Plan

1. System Philosophy

The "Bakery Ledger" is designed to be **user-simple but accounting-rigorous**. It uses a hidden Double-Entry Bookkeeping system to ensure every dollar is tracked via Debits and Credits, while the UI focuses on "Bakery Actions" (Selling, Buying, Paying).

2. Technical Stack

- **Frontend:** React (SPA)
- **Styling:** Bulma CSS (Responsive, Mobile-First)
- **Icons:** Solar Icon Set (Bold & Linear)
- **Backend/Database:** Firebase (Auth, Firestore)
- **Money Handling: Cents-only Integer logic** (e.g., $10.50 is stored as 1050). No floats.

3. Core Feature Set

A. Authentication & User Management

- **Admin-Only Creation:** New users can only be added by the "Admin" role via the internal dashboard.
- **Role-Based Access (RBAC):** * *Staff:* Log sales, log waste, view inventory.
    - *Admin:* Manage users, view P&L reports, delete/edit ledger entries.

B. The Transaction Engine (Double-Entry)

- **Automated Journaling:** When a user logs a "Sale," the system automatically creates a journal_entry document with balanced Debit/Credit lines.
- **Payment Status Tracking:** Tracks unpaid, partial, and paid statuses for both Accounts Receivable (Money customers owe you) and Accounts Payable (Money you owe suppliers).
- **Check Registry:** Track check numbers and "cleared" status for bank reconciliation.

C. Bakery-Specific Modules

- **Sales Tracker:** Quick-entry for daily bread and pastry sales.
- **Bill & Expense Logger:** Capture vendor invoices and categorize them (Ingredients, Rent, Utilities).
- **Outstanding Monitor:** High-visibility dashboard cards showing "Total Owed to Us" vs "Total We Owe."
- **Inventory & Waste:** Track raw materials and log "Burn/Unsold" items to adjust equity automatically.

4. Firestore Schema (Integrated)

Collection: accounts (Chart of Accounts)

Field	Type	Description
id	String	e.g., "1000_cash"
name	String	"Main Cash Drawer"
type	String	asset, liability, equity, revenue, expense
balance	Integer	Total cents (updated via Cloud Functions or UI logic)

Collection: journal_entries

Field	Type	Description
date	Timestamp	Transaction date
type	String	sale, purchase, payment, check, waste
status	String	unpaid, partial, paid
ref_no	String	Check #, Invoice #, or Receipt #
total_cents	Integer	The transaction face value
outstanding_cents	Integer	Amount remaining to be paid
lines	Array<Map>	[{account_id, debit, credit}] - Must sum to 0
metadata	Map	{created_by, vendor_name, customer_name, items}

5. UI/UX Principles

- **The Level Bar:** Top of screen shows Assets vs. Liabilities using Bulma's .level component.
- **Color Coding:** * has-text-success (Green) for Income/Cleared.
    - has-text-danger (Red) for Expenses/Overdue.
    - has-text-link (Blue) for Outstanding receivables.
- **Action Buttons:** Large, touch-friendly buttons with Solar Icons for mobile tablet use at the bakery counter.